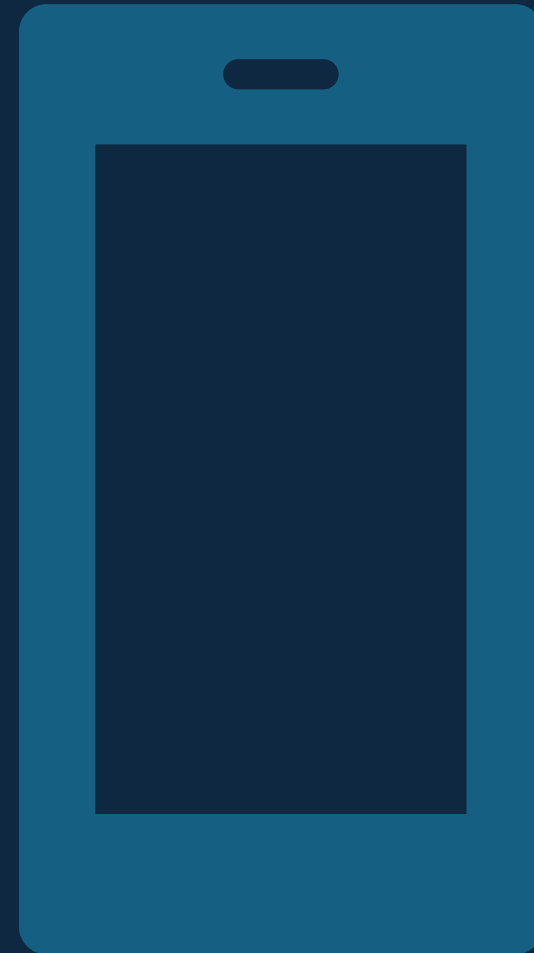


Javascript – BASE PARA O DESENVOLVIMENTO COM REACT

Material

MOBILE_001





Introdução

- JavaScript é uma linguagem de programação amplamente utilizada no desenvolvimento web.
- Ela permite criar interações dinâmicas em páginas web e desenvolver aplicativos completos.

JS

Histórico

- O JavaScript foi criado por Brendan Eich em 1995, enquanto trabalhava na Netscape Communications Corporation.
- Inicialmente, a linguagem foi concebida como uma forma de adicionar interatividade às páginas web, que na época eram estáticas e limitadas em suas funcionalidades.



Histórico

- Originalmente, a linguagem foi chamada de "Mocha", mas logo foi renomeada para "LiveScript" e finalmente para "JavaScript".
- O objetivo era capitalizar a popularidade da linguagem Java naquela época, embora o JavaScript não tenha relação direta com o Java.



Histórico 1996 - ...

Em 1996, a Microsoft lançou o Internet Explorer 3.0, com suporte para JavaScript. Isso levou à padronização da linguagem, e em 1997 foi lançada a primeira versão do padrão ECMAScript, que define a especificação da linguagem JavaScript.

ECMAScript é o nome oficial da linguagem, mas o termo JavaScript é amplamente utilizado para se referir a ela.



Histórico ... - Atual

Ao longo dos anos, a linguagem JavaScript passou por várias atualizações e adições de recursos. As principais versões do ECMAScript incluem o ES5 (lançado em 2009), ES6 (também conhecido como ES2015, lançado em 2015), ES7, ES8 e assim por diante.

Cada nova versão trouxe recursos e funcionalidades poderosos para a linguagem.

Atual: 2026 – 17ª Edição.

Popularidade

Nos últimos anos, o JavaScript se tornou uma das linguagens de programação mais populares e amplamente utilizadas, especialmente no desenvolvimento web.

Ela permite a criação de aplicativos web interativos, aplicações de uma única página (SPA), APIs e muito mais.

Além disso, com o surgimento de frameworks e bibliotecas populares como **ReactJS, AngularJS e Vue.js**, o JavaScript se tornou ainda mais dominante no desenvolvimento front-end. Essas ferramentas facilitam a criação de interfaces de usuário dinâmicas e interativas.



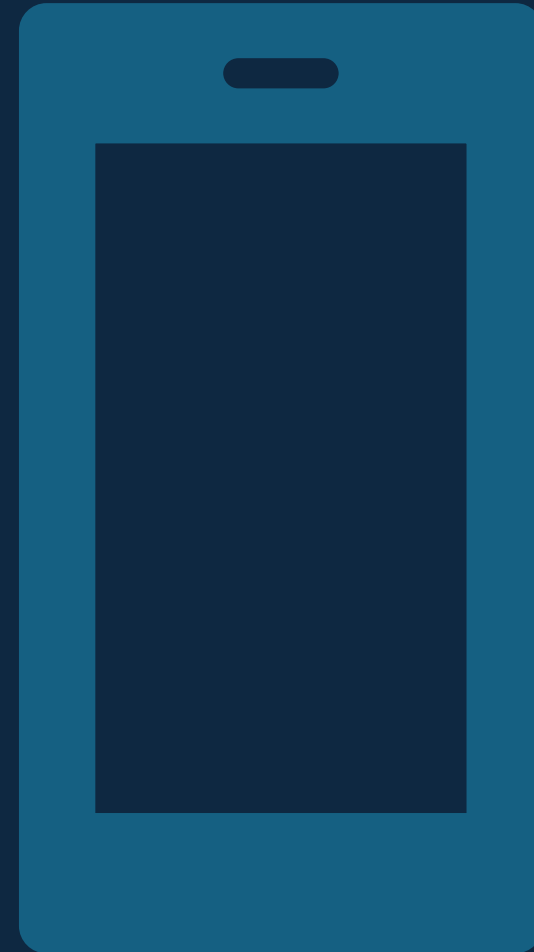


Nota

É importante ressaltar que o JavaScript não está limitado apenas ao ambiente do navegador. Com o advento do Node.js, ele também pode ser executado no servidor, permitindo o desenvolvimento de aplicativos web em tempo real e outras aplicações de back-end.

Conceitos Básicos

Material
MOBILE_001



Javascript – Tipos de Dados

- Em JavaScript, existem vários tipos de dados que podem ser atribuídos a uma variável. Os principais tipos de dados incluem:
 - **Números:** São valores numéricos, como 5, 3.14, -10, etc.
 - **Strings:** São sequências de caracteres, envolvidas por aspas simples (") ou aspas duplas ("). Por exemplo: "Olá, mundo!".
 - **Booleanos:** Representam os valores lógicos verdadeiro (true) ou falso (false).
 - **Arrays:** São coleções ordenadas de valores, que podem ser de diferentes tipos de dados. Os elementos de um array são acessados usando índices. Por exemplo: [1, 2, 3, 4].
 - **Objetos:** São estruturas de dados que armazenam propriedades e seus respectivos valores. Por exemplo: { nome: "João", idade: 25 }.

Variáveis

- Em JavaScript, as variáveis são usadas para armazenar valores.
- Podemos declarar variáveis usando as palavras-chave **let** ou **const**.
- A diferença entre elas é que **let** permite reatribuir um novo valor à variável, enquanto **const** cria uma variável com um valor fixo (constante).

Exemplo Variáveis

javascript

```
let nome = "João"; // Variável que pode ser reatribuída  
const idade = 25; // Variável constante (imutável)
```

Escopo

- Escopo: O escopo em JavaScript define a visibilidade e a acessibilidade de variáveis em diferentes partes do código. Existem dois principais tipos de escopo:
 - **Escopo global:** Variáveis declaradas fora de qualquer função são consideradas globais e podem ser acessadas em qualquer lugar do código.
 - **Escopo local:** Variáveis declaradas dentro de uma função são consideradas locais e só podem ser acessadas dentro dessa função.

Exemplo de Escopo

javascript

```
let nome = "João"; // Escopo global

function saudacao() {
  let mensagem = "Olá"; // Escopo local
  console.log(mensagem + ", " + nome);
}

saudacao();
```

Nota

- ▶ Para execução de um programa em Javascript em linha de comando (windows, linux ou mac). Precisamos instalar o programa node.
- ▶ Levamos em conta que aqui já temos o node instalado.
- ▶ Caso queira instalar em sua casa, o professor orientará como fazê-lo. É bem simples.

Exercícios para prática

► Tipos de dados:

1. Declare uma variável numero e atribua a ela um valor numérico.
2. Declare uma variável texto e atribua a ela uma string.
3. Declare uma variável verdadeiro e atribua a ela o valor booleano verdadeiro.
4. Declare uma variável numeros e atribua a ela um array com três números.
5. Declare uma variável pessoa e atribua a ela um objeto com propriedades nome e idade.

Exercícios para prática

➤ Escopo:

1. Crie uma função chamada `saudacao` que recebe um parâmetro `nome` e imprime uma saudação com o nome.
2. Chame a função `saudacao` passando um nome como argumento.
3. Declare uma variável `mensagem` dentro da função `saudacao` e tente acessá-la fora da função. O que acontece?

Mais prática

■ Desafio:

- a) Crie uma função chamada calculaMedia que recebe dois parâmetros nota1 e nota2 e retorna a média dessas duas notas.
- b) Chame a função calculaMedia passando duas notas como argumentos e imprima o resultado.

Javascript – Estruturas de Controle

Material
MOBILE_001





Conceitos

- Operadores
- Estruturas de Decisão
- Estruturas de Repetição

Operadores

- Em programação, os operadores são símbolos ou palavras-chave especiais que realizam operações em um ou mais valores, produzindo um resultado.
- Os operadores são fundamentais para executar cálculos, tomar decisões, comparar valores e realizar diversas outras manipulações nos dados dentro de um programa.

Tipos de operadores

➤ 1. Operadores Aritméticos:

- Esses operadores são utilizados para realizar operações matemáticas básicas, como adição, subtração, multiplicação, divisão e cálculos de resto. Eles permitem manipular números e obter resultados com base em fórmulas e expressões matemáticas.

javascript

```
let a = 5;
```

```
let b = 2;
```

```
console.log(a + b); // Resultado: 7 (adição)
```

```
console.log(a - b); // Resultado: 3 (subtração)
```

```
console.log(a * b); // Resultado: 10 (multiplicação)
```

```
console.log(a / b); // Resultado: 2.5 (divisão)
```

```
console.log(a % b); // Resultado: 1 (módulo)
```

```
console.log(a++); // Resultado: 5 (incremento)
```

```
console.log(b--); // Resultado: 2 (decremento)
```

Tipos de Operadores

➤ 2. Operadores de Atribuição:

- Esses operadores são usados para atribuir valores a variáveis. Eles permitem atualizar o valor de uma variável com base em operações aritméticas ou outros valores.

javascript

```
let x = 10;
```

```
x += 5; // Equivalente a x = x + 5
```

```
console.log(x); // Resultado: 15
```

```
let y = 8;
```

```
y -= 3; // Equivalente a y = y - 3
```

```
console.log(y); // Resultado: 5
```

Tipos de Operadores

➤ 3. Operadores Relacionais:

- Também conhecidos como operadores de comparação, eles são usados para comparar valores e verificar relações entre eles.
- Esses operadores retornam um valor booleano (verdadeiro ou falso) com base na comparação realizada. Exemplos incluem igualdade, desigualdade, maior que, menor que, entre outros.



```
let a = 5;  
let b = 3;  
let c = "5";
```

```
console.log(a == b);    // Resultado: false (igualdade)  
console.log(a != b);    // Resultado: true (desigualdade)  
console.log(a === c);   // Resultado: false (igualdade estrita)  
console.log(a !== c);   // Resultado: true (desigualdade estrita)  
console.log(a > b);     // Resultado: true (maior que)  
console.log(a < b);     // Resultado: false (menor que)  
console.log(a >= b);    // Resultado: true (maior ou igual a)  
console.log(a <= b);    // Resultado: false (menor ou igual a)
```

Tipos de Operadores

➤ 4. Operadores Lógicos:

- Esses operadores são utilizados para combinar expressões lógicas e realizar operações booleanas.
- Eles permitem tomar decisões com base em múltiplas condições, avaliar a veracidade de expressões complexas e criar lógica condicional em programas.

javascript



```
let a = 5;
```

```
let b = 3;
```

```
console.log(a > 0 && b > 0);    // Resultado: true (E lógico)  
console.log(a > 0 || b > 0);    // Resultado: true (OU lógico)  
console.log(!(a > 0));          // Resultado: false (NÃO lógico)
```

Tipos de Operadores

➤ 5. Operadores Bit a Bit:

- Esses operadores realizam operações em nível de bits em valores numéricos.
- Eles manipulam os bits individuais dos valores, permitindo ações como operações de máscara de bits, deslocamento de bits e operações de bit a bit.

javascript

```
let a = 5;    // Representado em binário como 101
let b = 3;    // Representado em binário como 011

console.log(a & b);    // Resultado: 1 (AND bit a bit)
console.log(a | b);    // Resultado: 7 (OR bit a bit)
```

Nota

- O operador **AND** bit a bit compara os bits correspondentes de dois valores e retorna um novo valor com **os bits setados para 1** somente quando os bits correspondentes **em ambos os valores são 1**. Caso **contrário, o bit resultante será 0**. Aqui está uma tabela que ilustra o funcionamento do operador AND bit a bit:

```
less

  5 (em binário: 101)
& 3 (em binário: 011)
-----
  1 (em binário: 001)
```

Nota 2

- O operador **OR** bit a bit compara os bits correspondentes de dois valores e retorna um novo valor com os bits setados para 1 sempre que pelo menos um dos bits correspondentes em ambos os valores for 1. Se ambos os bits correspondentes forem 0, o bit resultante será 0. Veja o exemplo abaixo:

```
less

    5 (em binário: 101)
|   3 (em binário: 011)
-----
    7 (em binário: 111)
```

Estruturas de decisão

- As estruturas de controle de decisão permitem que você tome decisões em seu código com base em condições específicas.
- Elas são usadas para executar diferentes blocos de código com base em testes lógicos.
- As principais estruturas de controle de decisão em JavaScript são **if**, **else if** e **else**.

Exemplo de if

javascript

```
let idade = 18;  
if (idade >= 18) {  
    console.log("Você é maior de idade.");  
}
```

Exemplo de else if

javascript

```
let idade = 15;
if (idade >= 18) {
    console.log("Você é maior de idade.");
} else if (idade >= 13) {
    console.log("Você é um adolescente.");
} else {
    console.log("Você é uma criança.");
}
```

Exemplo de else

javascript

```
let hora = 14;  
if (hora < 12) {  
    console.log("Bom dia!");  
} else if (hora < 18) {  
    console.log("Boa tarde!");  
} else {  
    console.log("Boa noite!");  
}
```

Estruturas de Repetição

- As estruturas de controle de repetição permitem que você execute um bloco de código várias vezes com base em uma condição específica. Elas são usadas quando você precisa repetir uma ação ou processar uma sequência de dados.
- As principais estruturas de controle de repetição em JavaScript são **while**, **do...while** e **for**.

Comando while

javascript

```
let contador = 1;
while (contador <= 5) {
  console.log(contador);
  contador++;
}
```

Comando do..while

javascript

```
let contador = 1;  
do {  
    console.log(contador);  
    contador++;  
} while (contador <= 5);
```

Comando for

- O **for** executa um bloco de código por um número específico de vezes. Ele consiste em três partes: inicialização, condição e atualização. Veja um exemplo:

javascript

```
for (let i = 1; i <= 5; i++) {  
    console.log(i);  
}
```

Nota

- Uma observação importante sobre as estruturas de controle é a **necessidade de ter cuidado com a criação de loops infinitos**. Em loops infinitos, o código fica preso em um ciclo contínuo e não consegue sair dele, o que pode causar travamento do programa.
- Para evitar loops infinitos, **é fundamental garantir que a condição de parada seja alcançada em algum momento**. Certifique-se de que a condição do loop seja atualizada corretamente em cada iteração para evitar que o loop seja executado indefinidamente.

Exercícios para prática

1. Verificar se um número é par ou ímpar.
2. Verificar se uma pessoa é adulta (idade maior ou igual a 18) e se possui carteira de motorista.
3. Verificar se uma palavra é um palíndromo (se pode ser lida da mesma forma de trás para frente).
4. Verificar se um ano é bissexto.
5. Verificar se um número é positivo, negativo ou zero.
6. Classificar um estudante com base em sua pontuação em uma prova (A, B, C, D ou F).
7. Verificar se uma senha digitada pelo usuário é válida (deve ter pelo menos 8 caracteres e conter números e letras).
8. Verificar se um número é divisível por outro número.
9. Verificar se um triângulo é equilátero, isósceles ou escaleno com base em seus lados.
10. Verificar se um número é primo.

Javascript – Funções

Material
MOBILE_001





Funções

- Funções. O que são ?
- Estrutura das funções.
- Exemplos.
- Exercícios.

Funções

- As funções em JavaScript são blocos de código que podem ser definidos e reutilizados para executar tarefas específicas.
- Elas ajudam a organizar e modularizar o código, permitindo que você divida seu programa em partes menores e mais gerenciáveis.
- Ao utilizar funções, você pode escrever um trecho de código uma vez e chamá-lo várias vezes em diferentes partes do seu programa, evitando repetição e melhorando a legibilidade e a manutenção do código.

Estrutura das funções

```
function nomeDaFuncao(parametro1, parametro2) {  
    // Corpo da função - Bloco de código executado quando a função é chamada  
    // Pode conter instruções, cálculos, lógica, estruturas de controle e muito mais  
    // Pode também retornar um valor usando a instrução 'return'  
}
```

Exemplo

- ▶ Vamos ver um exemplo simples de uma função que recebe dois números como parâmetros e retorna a soma deles:

javascript

```
function somar(a, b) {  
    return a + b;  
}  
  
// Chamando a função e exibindo o resultado  
console.log(somar(3, 4)); // Resultado: 7
```

Exemplo da função somar

- No exemplo, a função somar recebe dois parâmetros **a** e **b**.
- Dentro do corpo da função, os parâmetros são utilizados para realizar a soma dos números.
- O resultado é retornado usando a instrução **return**.
- Em seguida, a função é chamada e o resultado é exibido no console.

Exemplo função fatorial

- Agora, vamos ilustrar a utilização de funções na resolução de algoritmos e incorporar lógica com estruturas de controle.
- Podemos trabalhar com um exemplo simples, como o cálculo do fatorial de um número.

javascript

```
function calcularFatorial(n) {  
    if (n === 0 || n === 1) {  
        return 1;  
    } else {  
        return n * calcularFatorial(n - 1);  
    }  
}
```

```
// Chamando a função para calcular o fatorial de 5  
let resultado = calcularFatorial(5);  
console.log(resultado); // Resultado: 120
```

Fatorial recursivo

- Nesse exemplo, a função `calcularFatorial` utiliza uma estrutura de controle **if...else** para verificar se o número `n` é igual a 0 ou 1.
- Se for, retorna 1, pois o fatorial de 0 e 1 é sempre 1.
- Caso contrário, utiliza a recursividade para calcular o fatorial de `n` multiplicando-o pelo fatorial de `n - 1`.
- Dessa forma, a função é chamada repetidamente até que `n` atinja 0 ou 1, retornando o resultado final.

Exercícios para prática

1. Escreva uma função que recebe dois números como parâmetros e retorna o maior deles.
2. Crie uma função que recebe um número como parâmetro e retorna verdadeiro se ele for par, ou falso se for ímpar.
3. Implemente uma função que recebe um array de números como parâmetro e retorna a soma de todos os elementos.
4. Faça uma função que recebe uma string como parâmetro e retorna a quantidade de vogais presentes nessa string.
5. Crie uma função que recebe um número inteiro positivo como parâmetro e retorna a sequência de Fibonacci correspondente a esse número (sem utilizar recursão)

Exercícios para prática

6. Escreva uma função que recebe um array de palavras como parâmetro e retorna um novo array contendo somente as palavras que começam com a letra "A".
7. Implemente uma função que recebe um número inteiro como parâmetro e verifica se ele é um número primo.
8. Faça uma função que recebe um array de números como parâmetro e retorna o produto de todos os elementos.
9. Crie uma função que recebe um número inteiro positivo como parâmetro e retorna o reverso desse número (por exemplo, 12345 se torna 54321).
10. Escreva uma função que recebe um array de strings como parâmetro e retorna a quantidade total de caracteres em todas as strings.

Javascript – Funções – Outros Conceitos

Material
MOBILE_001



Escopo de Funções

- Ao entender o escopo, você adquire a capacidade de escrever código mais eficiente, legível e livre de erros.

Razões para compreender (escopo)

1. **Evitar conflitos de nomes:** Ao dividir um programa em diferentes funções, é provável que você precise usar variáveis com o mesmo nome em contextos diferentes.
2. **Encapsulamento de variáveis:** O escopo permite encapsular variáveis dentro de funções. Isso significa que essas variáveis só são acessíveis dentro das funções em que foram declaradas.
3. **Reutilização de nomes:** Entender o escopo permite reutilizar nomes de variáveis em diferentes partes do código sem gerar conflitos.
4. **Segurança e privacidade de dados:** Ao limitar o acesso às variáveis em escopos específicos, você aumenta a segurança e a privacidade de dados.
5. **Otimização de memória:** O escopo também está relacionado à eficiência do programa. Quando uma função é concluída, as variáveis locais dentro dela são liberadas da memória.

Variáveis Globais e Locais

- Variáveis globais e locais:
 - No **escopo global**, as variáveis são declaradas fora de qualquer função e podem ser acessadas em qualquer lugar do código.
 - No **escopo local**, as variáveis são declaradas dentro de uma função e só podem ser acessadas dentro dessa função.

Exemplo

javascript

```
let nome = "João"; // Variável global
```

```
function saudacao() {
```

```
    let mensagem = "Olá"; // Variável local
```

```
    console.log(mensagem + ", " + nome); // Acessa variáveis t
```

```
}
```

```
saudacao(); // Resultado: "Olá, João"
```

Locais e Globais

- Nesse exemplo, a variável **nome** é uma **variável global**, pois foi declarada fora da função.
- Ela pode ser acessada tanto **dentro da função saudacao quanto fora dela**.
- Já a variável **mensagem** é uma **variável local**, pois foi declarada dentro da função saudacao e **só pode ser acessada dentro dessa função**.

Aninhamento

- Quando uma função é declarada dentro de outra função, ela cria um **escopo aninhado**.
- As variáveis **declaradas na função externa podem ser acessadas pela função interna**, mas não o contrário.

Aninhamento

javascript

```
function calcularIdade() {  
    let idade = 25; // Variável na função externa  
  
    function exibirIdade() {  
        console.log(idade); // Acessa a variável da função externa  
    }  
  
    exibirIdade(); // Resultado: 25  
}  
  
calcularIdade();
```

Sombreamento

- **Sombreamento de variáveis:** Quando uma variável é declarada em um escopo local com o mesmo nome de uma variável em um escopo externo, ocorre o sombreamento da variável externa. Isso significa que a variável local tem prioridade sobre a variável externa dentro do escopo local.

javascript

 Copy code

```
let nome = "Maria"; // Variável global

function saudacao() {
  let nome = "João"; // Variável local com o mesmo nome

  console.log("Olá, " + nome); // Acessa a variável local
}

saudacao(); // Resultado: "Olá, João"
console.log("Nome global: " + nome); // Resultado: "Nome global: Maria"
```

Sombreamento

- Nesse exemplo, a função saudacao **possui uma variável local chamada nome, que sombreia a variável global de mesmo nome.**
- Dentro da função, a variável local é usada para exibir a saudação, enquanto fora da função, a variável global é acessada.



let e var

- **let:** A declaração let introduz uma variável com escopo de bloco, o que significa que ela é visível apenas dentro do bloco onde foi declarada. Um bloco é delimitado por chaves {}. Variáveis declaradas com let têm escopo limitado ao bloco em que foram definidas, incluindo loops, condicionais e funções.
- **var:** A declaração var, por outro lado, tem escopo de função ou escopo global. Isso significa que uma variável declarada com var é visível dentro da função onde foi declarada ou, se não estiver dentro de uma função, é visível globalmente.

let

```
function exemplo() {  
  if (true) {  
    let x = 10; // Escopo de bloco  
    console.log(x); // 10  
  }  
  console.log(x); // ReferenceError: x is not defined (não está acessível)  
}
```



var

```
function exemplo() {  
  if (true) {  
    var y = 20; // Escopo de função  
    console.log(y); // 20  
  }  
  console.log(y); // 20 (ainda está acessível fora do bloco)  
}
```

Redeclaração

- **let:** A redeclaração de uma variável com let no mesmo escopo não é permitida. Tentar redeclarar uma variável com let resultará em um erro.
- **var:** A redeclaração de uma variável com var no mesmo escopo é permitida, mas pode levar a comportamentos inesperados, pois a variável será substituída pela nova declaração.

javascript

 Copy code

```
for (let i = 0; i < 3; i++) {  
  setTimeout(function() {  
    console.log(i); // 0, 1, 2 (cada iteração cria um novo escopo)  
  }, 1000);  
}
```

```
for (var j = 0; j < 3; j++) {  
  setTimeout(function() {  
    console.log(j); // 3, 3, 3 (mesma instância compartilhada)  
  }, 1000);  
}
```

Explicação

- No exemplo com **let**, cada iteração do loop cria um novo escopo para a variável **i**, permitindo que cada função de retorno de `setTimeout` acesse o valor correto de **i** em sua respectiva iteração.
- Já no exemplo com **var**, a mesma instância de **j** é compartilhada entre todas as funções de retorno de `setTimeout`, resultando no mesmo valor para todas as iterações.



Resumo da Aula

- Nesta aula, vimos:
 - Introdução: Histórico do JavaScript, sua evolução (ECMAScript) e versatilidade entre Frontend e Backend (Node.js).
 - Conceitos Básicos: Tipos de dados, armazenamento em variáveis (let e const) e noções de escopo.
 - Estruturas de Controle: Operadores aritméticos, relacionais, lógicos, bit a bit e lógica de decisão (if/else) e repetição (for/while).
 - Funções: Definição, estrutura, escopo (global/local), aninhamento, recursividade e a diferença entre let e var.

Fim



Prof. Maromo